

Concepts

Rust

Nautilus has a complete Rust implementation under the `crates/` directory. You can write actors, strategies, run backtests, and trade live without Python. The domain model is shared across all paths, and the v2 PyO3 path runs Python strategies on the Rust engine directly.

⚠ The Rust API is under active development. Method signatures and trait requirements may change between releases.

System implementations

Nautilus has three implementations. Understanding where each stands helps you choose the right one for your use case.

- **v1 legacy:** Cython/Python classes under `nautilus_trader/`. Fully featured with the broadest component coverage.
- **v2 Rust:** Pure Rust under `crates/`. Runs without Python.
- **v2 PyO3:** Python user-components (actors, strategies) running on the Rust core via PyO3 bindings. Combines Python convenience with Rust engine performance.

Capability matrix

Component	v1 legacy (Cython)	v2 Rust	v2 PyO3 (Python on Rust)
Strategy	✓	✓	✓
Actor	✓	✓	✓
DataEngine	✓	✓	✓
ExecutionEngine	✓	✓	✓

Component	v1 legacy (Cython)	v2 Rust	v2 PyO3 (Python on Rust)
RiskEngine	✓	✓	✓
BacktestEngine	✓	✓	✓
BacktestNode	✓	✓	✓
LiveNode	✓	✓	✓
OrderEmulator	✓	✓	✓
Matching engine	✓	✓	✓
Portfolio	✓	✓	✓
Accounts	✓	✓	✓
Cache	✓	✓	✓
MessageBus	✓	✓	✓
Data catalog	✓	✓	✓
Indicators	✓	✓	✓
Exec algorithms	TWAP	TWAP	TWAP
Controller	✓	-	-
Tearsheets	✓	-	-
Config serialization	✓	-	-

Adapters

Adapter	v1 legacy (Cython)	v2 Rust	v2 PyO3
Architect AX	✓	✓	✓
Betfair	✓	✓	✓
Binance	✓	✓	✓

Adapter	v1 legacy (Cython)	v2 Rust	v2 PyO3
BitMEX	✓	✓	✓
Bybit	✓	✓	✓
Databento	✓	✓	✓
Deribit	✓	✓	✓
dYdX	✓	✓	✓
Hyperliquid	✓	✓	✓
Interactive Brokers	✓	-	-
Kraken	✓	✓	✓
OKX	✓	✓	✓
Polymarket	✓	✓	✓
Sandbox	✓	✓	✓
Tardis	✓	✓	✓

Choosing a path

- **v1 legacy** is the most complete today. Use it if you need the Controller, tearsheets, Interactive Brokers, or config serialization.
- **v2 Rust** gives native performance without a Python runtime. All core trading functionality is available. Use it for latency-sensitive deployments or teams that prefer a compiled language.
- **v2 PyO3**: Python user-components (actors, strategies) run on the Rust core engine with Rust performance for data processing and execution, while keeping the Python authoring experience.

Project setup

The Nautilus crates are published to crates.io. Add them to your `Cargo.toml` :

```
[dependencies]
nautilus-backtest = "0.55"
nautilus-common = "0.55"
nautilus-execution = "0.55"
nautilus-model = { version = "0.55", features = ["stubs"] }
nautilus-trading = { version = "0.55", features = ["examples"] }

anyhow = "1"
log = "0.4"
```



For live trading, add the live crate and the adapter for your venue:

```
[dependencies]
nautilus-live = "0.55"
nautilus-okx = "0.55"
```



To track the latest development branch, point all Nautilus dependencies at the same git source to avoid type mismatches between crates.io and git versions:

```
[dependencies]
nautilus-backtest = { git = "https://github.com/nautechsystems/nautilus_trader.git", branch = "devel" }
nautilus-common = { git = "https://github.com/nautechsystems/nautilus_trader.git", branch = "devel" }
nautilus-execution = { git = "https://github.com/nautechsystems/nautilus_trader.git", branch = "devel" }
nautilus-model = { git = "https://github.com/nautechsystems/nautilus_trader.git", branch = "devel" }
nautilus-trading = { git = "https://github.com/nautechsystems/nautilus_trader.git", branch = "devel" }
```



The minimum supported Rust version (MSRV) is **1.95.0**.

Feature flags

Flag	Crate	Effect
<code>high-precision</code>	<code>nautilus-model</code>	16-digit fixed precision (default is 9). Required for crypto.
<code>stubs</code>	<code>nautilus-model</code>	Test instrument stubs (<code>audusd_sim</code> , etc.).
<code>examples</code>	<code>nautilus-trading</code>	Example strategies (<code>EmaCross</code> , <code>GridMarketMaker</code>).
<code>streaming</code>	<code>nautilus-backtest</code>	Catalog-based data streaming via <code>BacktestNode</code> .
<code>defi</code>	<code>nautilus-model</code>	DeFi data types. Implies <code>high-precision</code> .



Standard 9-digit precision handles most traditional finance instruments. Enable `high-precision` for crypto venues where prices can have many decimal places (e.g. `0.00000001`).

Actors

An actor receives market data, custom data/signals, and system events but does not manage orders. Implement the `DataActor` trait and bind your struct to `DataActorCore` via `Deref` / `DerefMut`. Your struct must also implement `Debug` (required by the blanket `Component` impl). The core provides subscription methods, cache access, and clock access directly on your struct.

Handler methods

Override any handler on the `DataActor` trait to receive the corresponding data or event. All handlers have default no-op implementations, so you only override what you need.

Handler	Receives
<code>on_start</code>	Actor started.
<code>on_stop</code>	Actor stopped.
<code>on_quote</code>	<code>QuoteTick</code>
<code>on_trade</code>	<code>TradeTick</code>
<code>on_bar</code>	<code>Bar</code>
<code>on_book_deltas</code>	<code>OrderBookDeltas</code>
<code>on_book</code>	<code>OrderBook</code> (at interval)
<code>on_instrument</code>	<code>InstrumentAny</code>
<code>on_mark_price</code>	<code>MarkPriceUpdate</code>
<code>on_index_price</code>	<code>IndexPriceUpdate</code>
<code>on_funding_rate</code>	<code>FundingRateUpdate</code>
<code>on_option_greeks</code>	<code>OptionGreeks</code>

Handler	Receives
<code>on_option_chain</code>	<code>OptionChainSlice</code>
<code>on_instrument_status</code>	<code>InstrumentStatus</code>
<code>on_order_filled</code>	<code>OrderFilled</code>
<code>on_order_canceled</code>	<code>OrderCanceled</code>
<code>on_time_event</code>	<code>TimeEvent</code>

For a step-by-step walkthrough, see the [Write an Actor \(Rust\)](#) how-to guide. For a complete example, see [BookImbalanceActor](#).

Strategies

A strategy extends an actor with order management. Implement both `DataActor` (for data handling) and `Strategy` (for access to `StrategyCore`). The `StrategyCore` wraps `DataActorCore` and adds an `OrderFactory`, `OrderManager`, and portfolio integration.

Order management

The `Strategy` trait provides order methods through `StrategyCore`:

Method	Action
<code>submit_order</code>	Submit a new order to the venue.
<code>submit_order_list</code>	Submit a list of contingent orders.
<code>modify_order</code>	Modify price, quantity, or trigger price.
<code>cancel_order</code>	Cancel a specific order.
<code>cancel_orders</code>	Cancel a filtered set of orders.
<code>cancel_all_orders</code>	Cancel all orders for an instrument.
<code>close_position</code>	Close a position with a market order.

Method	Action
<code>close_all_positions</code>	Close all open positions.

The `OrderFactory` (accessed via `self.core.order_factory()`) builds order objects: `market`, `limit`, `stop_market`, `stop_limit`, `market_if_touched`, `limit_if_touched`, and `trailing_stop_market`.

For a step-by-step walkthrough, see the [Write a Strategy \(Rust\)](#) how-to guide. For complete examples, see [EmaCross](#) and [GridMarketMaker](#).

Running Rust components

Rust strategies and actors can run through three paths. The examples below use strategies, but the same pattern applies to actors via `add_actor` (pure Rust) and `add_native_actor` (from Python).

Pure Rust

Write your strategy and `main` function in Rust, then build a standalone binary with `cargo build`. This path requires no Python runtime.

```
let strategy = GridMarketMaker::new(config);
node.add_strategy(strategy)?;
node.run().await?;
```



See [Run Live Trading \(Rust\)](#) for a full walkthrough.

Native config from Python

Pass a config to `add_native_strategy` to register a built-in Rust strategy from Python. The Rust side constructs the strategy and registers it with the engine. Python provides the configuration; all execution happens in Rust.

```
from nautilus_trader.core.nautilus_py3.trading import GridMarketMakerConfig

config = GridMarketMakerConfig(
    instrument_id=InstrumentId.from_str("BTC-USDT-SWAP.OKX"),
    max_position=Quantity.from_str("10.0"),
    trade_size=Quantity.from_str("0.1"),
    num_levels=5,
    grid_step_bps=15,
)
```



```
node.add_native_strategy(config)
```

Built-in strategy configs:

Config	Strategy
<code>EmaCrossConfig</code>	<code>EmaCross</code>
<code>GridMarketMakerConfig</code>	<code>GridMarketMaker</code>
<code>DeltaNeutralVolConfig</code>	<code>DeltaNeutralVol</code>

Built-in actor configs (via `add_native_actor`):

Config	Actor
<code>BookImbalanceActorConfig</code>	<code>BookImbalanceActor</code>

Users who compile from source can add their own components to this path. Add a `#[pyclass]` config and a dispatch arm in `add_native_strategy` or `add_native_actor`. The component then works from Python without PyO3 wrappers on the type itself.

Plugin loading (planned)

A future plugin system will load compiled shared libraries at runtime. Users compile strategies and actors as `cdylib` crates and the node loads them without recompilation. This path is not yet available.

Backtesting

For annotated walkthroughs of both APIs, see the [Run a Backtest \(Rust\)](#) how-to guide.

`BacktestEngine` (low-level API)

Construct the engine, add venues and instruments, load data, register strategies, and run. See the full working example:

```
cargo run -p nautilus-backtest --features examples --example engine-ema-cross
```



Source: [crates/backtest/examples/engine_ema_cross.rs](#)

BacktestNode (high-level API)

Loads data from a `ParquetDataCatalog` and supports streaming in configurable chunk sizes. Requires the `streaming` feature on `nautilus-backtest`. See the full working example:

```
cargo run -p nautilus-backtest --features examples,streaming --example node-ema-cross
```



Source: [crates/backtest/examples/node_ema_cross.rs](#)

Live trading

For an annotated walkthrough, see the [Run Live Trading \(Rust\)](#) how-to guide.

The `LiveNode` connects to real venues through adapter clients. The builder pattern configures data and execution clients, then `run()` starts the async event loop. Each adapter provides its own factory and config types.

Adapter	Example
Architect AX	crates/adapters/architect_ax/examples/
Betfair	crates/adapters/betfair/examples/
Binance	crates/adapters/binance/examples/
BitMEX	crates/adapters/bitmex/examples/
Blockchain	crates/adapters/blockchain/examples/
Bybit	crates/adapters/bybit/examples/
Databento	crates/adapters/databento/examples/
Deribit	crates/adapters/deribit/examples/
dYdX	crates/adapters/dydx/examples/
Hyperliquid	crates/adapters/hyperliquid/examples/

Adapter	Example
Kraken	<code>crates/adapters/kraken/examples/</code>
OKX	<code>crates/adapters/okx/examples/</code>
Polymarket	<code>crates/adapters/polymarket/examples/</code>
Sandbox	<code>crates/adapters/sandbox/examples/</code>
Tardis	<code>crates/adapters/tardis/examples/</code>

Most adapters include `node_data_tester.rs` and `node_exec_tester.rs` examples. These test data requests, streaming, and order execution against live venues.

Related guides

- [Write an Actor \(Rust\)](#) - Step-by-step actor walkthrough.
- [Write a Strategy \(Rust\)](#) - Step-by-step strategy walkthrough.
- [Run a Backtest \(Rust\)](#) - BacktestEngine and BacktestNode usage.
- [Run Live Trading \(Rust\)](#) - LiveNode setup and venue connection.
- [Architecture](#) - System design and data/execution flow.
- [Actors](#) - Actor concepts (applies to both Python and Rust).
- [Strategies](#) - Strategy concepts and handler reference.
- [Events](#) - Event types and handler dispatch.
- [Backtesting](#) - Backtest concepts and matching engine behavior.

[< Adapters](#)[Previous Page](#)[DST >](#)[Next Page](#)